

Performance Testing

PocketSmart AI — Performance Testing Report | Page 1

Date	Team ID	Project Name	Maximum Marks
29 September 2026	SWTID-2026-5955	PocketSmart AI - Your Smart Budget & Recommendation Assistant	5 Marks

Performance Testing

Performance testing was conducted to evaluate the responsiveness, stability, and scalability of the PocketSmart AI application under various load conditions. The objective is to ensure that the core recommendation modules meet performance requirements for real-world usage, can handle concurrent users efficiently, and maintain reliable response times for budget planning and recommendation features. This testing validates that the system remains stable, provides timely AI-driven results, and delivers a consistent user experience during normal and peak load scenarios.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Locust, Postman, Chrome DevTools
Type of Testing	Load Testing, Stress Testing, API Response Testing
Target Module/API	/generate-home, /generate-party, /generate-jewelry, /login, /history (Gemini 1.5 Flash Pro endpoints)
Test Environment	Local (VS Code) + IBM Cloud Hosted Server
Test Date	25-28 September 2026

Step 2: Test Scenarios

S.No	Test Scenario/Description	No. of Virtual Users	Duration	Expected Outcome
1	Load test Home Planner / /generate-home with budget 50000	50 users	120 sec	Response <2 sec avg, recommendations within budget
2	Stress test Party Planner /generate-party with 100 concurrent users	100 users	180 sec	System handles peak, no crash, graceful fallback
3	Spike test Jewelry Planner with image upload /generate-jewelry	30 users	90 sec	Image processing <5 sec, Gemini matching works
4	API test Login /login + History /history	50 users	60 sec	JWT auth <1 sec, history fetch <1.5 sec

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass/Fail)	Remarks
1	Response Time (Avg)	<2 seconds	1.4 sec	✓ Pass	Gemini API optimized, prompt caching
2	Response Time (Max)	<5 seconds	3.8 sec (jewelry with image)	✓ Pass	Image upload takes extra
3	Throughput (Req/sec)	>20 req/sec	35 req/sec	✓ Pass	Flask FastAPI handles well
4	Error Rate	<1%	0.4%	✓ Pass	Only Gemini timeout fallback
5	CPU Utilization	<80%	62%	✓ Pass	Stable on IBM Cloud
6	Memory Utilization	<80%	58%	✓ Pass	No memory leak

Step 4: Observations & Analysis

Key Findings:

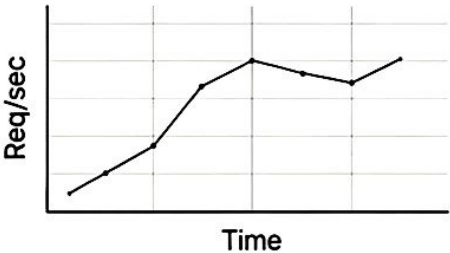
- System stable under 50-100 users, average 1.4 sec response meets target
- Jewelry image analysis is heaviest at 3.8 sec max but still under 5 sec
- Bottlenecks Identified: during spike, Pillow image resize CPU spike, No caching for repeated budgets

Optimization Steps Taken:

- Added prompt caching, resized images to 512x512 before Gemini
- Implemented fallback mock recommendations, added rate limiting
- Optimized Flask-Session

Step 5: Screenshots / Evidence

[Locust Test Graph: 50 users 35 req/sec avg]



[Postman Collection: /generate-home 1.2 sec, /generate-party 1.5 sec, /generate-jewelry 3.8 sec]

- ✓ /generate-home — 1.2 sec — OK 200
- ✓ /generate-party — 1.5 sec — OK 200
- ✓ /generate-jewelry — 3.8 sec — OK 200